

Examen de Java: Manejo de Ficheros con POO y Colecciones

¡Ojooo, mi primooo! Aquí tienes el examen de Java, pa' que demuestres que tienes más arte programando que un gitano tocando la guitarra en una cueva de Granada.

Instrucciones:

- Escribe tu código con compás y sin chapuzas.
 - Usa buenas prácticas, que aquí no queremos código que parezca una feria desordená.
 - Responde con alegría, pero sin copiar, que aquí se valora el esfuerzo, no el canteo.
-

Ejercicio Único: El Flamenco Digital

Vas a crear un programa en Java que maneje ficheros utilizando POO, `ArrayList` y `HashMap`. Para ello, sigue estos pasos:

1. **Crea una clase `FlamencoArchivo`** que maneje un fichero `flamenco.txt`.
 - Debe tener métodos para crear el archivo, escribir en él, leerlo y borrarlo.
 2. **Crea una clase `GestorFlamenco`** que gestione múltiples frases de flamenco usando un `ArrayList<String>`.
 - Debe permitir añadir frases nuevas y guardarlas en el fichero sin borrar las anteriores.
 3. **Usa un `HashMap<Integer, String>`** para almacenar frases con un identificador único.
 - Implementa métodos para añadir, recuperar y listar las frases con su ID.
 4. **Implementa en el `main`** la funcionalidad para:
 - Crear el fichero si no existe.
 - Añadir frases al `ArrayList` y al `HashMap`.
 - Guardar el contenido en el fichero.
 - Leer y mostrar el contenido del fichero con un mensaje tipo: *"Este archivo tiene X líneas de puro arte"*.
 - Borrar el fichero con confirmación.
-

¡Dale fuerte, que el código salga fino y con compás! TacaTa 🐎🔥🐎

Examen de Java: El código de los archivos encantados

Instrucciones

En este examen, demostrarás tu maestría en el manejo de ficheros de texto en Java, así como en el uso de programación orientada a objetos y colecciones. Sigue con atención los designios del enunciado y labra un código digno de los escribas del reino.

- Has de escribir código limpio, sin enredos ni despropósitos, pues la claridad es virtud de todo buen programador.
 - Absténete de copiar de ajenos pergaminos, que el conocimiento propio es mayor tesoro que el oro de los dragones.
 - Usa las herramientas del lenguaje sabiamente, tal como un caballero maneja su acero.
-

Ejercicio Único: El código de los archivos encantados

Cuenta la leyenda que en los vastos salones de la Gran Biblioteca de Eldoria se halla un código encantado que contiene los más antiguos relatos de la fantasía medieval. Tu cometido es desarrollar un programa en Java que administre este código, permitiendo la escritura, lectura y gestión de sus páginas.

1. Creación del código

Crea una clase **CodiceEncantado** que represente el código de relatos. Esta clase deberá manejar un fichero llamado **codice.txt** y poseer los siguientes métodos:

- **crearCodice()** → Crea el fichero si aún no existe.
- **escribirRelato(String relato)** → Añade un nuevo relato al código sin borrar los anteriores.
- **leerCodice()** → Muestra el contenido del código con un mensaje similar a:
"Oh, viajero, este código contiene X relatos de antaño."
- **borrarCodice()** → Borra el código previa confirmación del usuario.

2. El escriba real

Crea una clase **EscribaReal**, cuya función es gestionar los relatos antes de ser inscritos en el código. Para ello, hará uso de:

- Un **ArrayList** que contendrá los relatos pendientes de ser añadidos al código.
- Un **HashMap<Integer, String>** que almacenará los relatos con su identificador único.

Dicha clase deberá contar con los siguientes métodos:

- **añadirRelato(String relato)** → Agrega un relato al ArrayList y al HashMap.
- **listarRelatos()** → Muestra todos los relatos con su identificador.
- **guardarRelatosEnCodice()** → Escribe en el fichero todos los relatos del ArrayList, conservando su contenido previo.

3. El código en acción

En el **main**, el programa deberá permitir lo siguiente:

1. Verificar si el código existe y crearlo si es necesario.
2. Permitir al usuario añadir relatos al ArrayList y al HashMap.
3. Guardar estos relatos en el fichero sin perder los anteriores.
4. Leer el contenido del código y mostrarlo en pantalla con el mensaje indicado.
5. Brindar la opción de borrar el código si el usuario así lo desea.

Que el destino guíe tu código y la lógica sea tu aliada en esta prueba. ¡Demuestra tu valía y deja constancia de tu destreza en los anales del reino!

Ejercicio: Procesar un fichero de registros y generar un informe

Descripción:

Imagina que tienes un fichero de texto llamado `registros.txt` en el cual cada línea contiene un **registro** con el siguiente formato:

```
ID: <número de ID> | Nombre: <nombre de usuario> | Edad: <número> | Ciudad: <ciudad>
```

Tu tarea es escribir un programa que realice las siguientes acciones:

1. Leer el fichero `registros.txt` y crear un nuevo fichero llamado `procesado.txt`.
2. En el nuevo fichero `procesado.txt`, debe escribir:
 - El **ID** y **nombre de usuario** de cada línea.
 - Solo aquellos registros que cumplan con las siguientes condiciones:
 - La **edad** sea mayor o igual a 18.
 - La **ciudad** sea "Madrid" o "Barcelona".
3. Después de procesar los registros, el programa debe:
 - **Contar el número total de registros** que cumplen con los criterios anteriores y escribir este número al final del fichero.
4. Por último, debe generar un **informe en consola** que muestre el total de registros que cumplen con los criterios, además de listar los **nombres de los usuarios** que cumplen con la condición de edad y ciudad.

Ejemplo de `registros.txt`:

ID: 101 | Nombre: Juan Pérez | Edad: 20 | Ciudad: Madrid

ID: 102 | Nombre: Ana García | Edad: 17 | Ciudad: Barcelona

ID: 103 | Nombre: Pedro López | Edad: 25 | Ciudad: Valencia

ID: 104 | Nombre: Laura Fernández | Edad: 30 | Ciudad: Madrid

ID: 105 | Nombre: Marta Gómez | Edad: 22 | Ciudad: Barcelona

Salida esperada en `procesado.txt`:

ID: 101 | Nombre: Juan Pérez

ID: 104 | Nombre: Laura Fernández

ID: 105 | Nombre: Marta Gómez

Total de registros procesados: 3

Informe en consola:

Total de registros procesados: 3

Usuarios procesados:

- Juan Pérez
 - Laura Fernández
 - Marta Gómez
-

Instrucciones:

- Usa `BufferedReader` para leer el fichero `registros.txt`.
- Usa `BufferedWriter` para escribir el fichero `procesado.txt`.
- Asegúrate de **verificar** que el fichero de entrada (`registros.txt`) existe antes de procesarlo.
- El **formato del fichero de entrada** es consistente, pero deberías manejar correctamente los posibles errores o registros mal formateados.
- El programa debe **manejar excepciones** adecuadamente (por ejemplo, si el fichero no se encuentra, o si hay un formato inesperado).

Pistas:

- Usa expresiones regulares para extraer las partes del registro (ID, nombre, edad, ciudad).
- **Filtra** los registros de acuerdo con los criterios de edad y ciudad.
- **Cuenta** los registros que cumplen con los criterios y **muestra un informe en consola**

Imagina que trabajas en una empresa que gestiona una biblioteca digital. En dicha biblioteca, se encuentran varios documentos que representan libros, y todos estos libros están almacenados en ficheros de texto. Los ficheros tienen un formato sencillo donde el título del libro aparece en la primera línea, y el resto del fichero contiene la descripción del libro.

Tu tarea será desarrollar un programa en Java que permita realizar las siguientes operaciones sobre estos ficheros:

1. Comprobar la existencia y las propiedades de los ficheros y directorios.
2. Leer ficheros de texto de forma eficiente con `FileReader` y `BufferedReader`.
3. Escribir y modificar ficheros de texto con `FileWriter` y `BufferedWriter`.
4. Copiar el contenido de un fichero a otro utilizando únicamente `FileReader` y `FileWriter`.
5. Realizar otras operaciones comunes como renombrar, mover, borrar y listar el contenido de directorios.

Operaciones a realizar:

1. Comprobación de ficheros y directorios:
 - El programa debe comprobar si existe un directorio donde se guardan los libros.
 - Si el directorio no existe, debe crearlo automáticamente.
 - El programa debe verificar si el fichero de un libro existe antes de intentar leerlo. Si no existe, debe mostrar un mensaje indicando que el fichero no está disponible.
2. Leer ficheros de texto:
 - El programa debe leer los ficheros de texto que contienen información sobre los libros.
 - Utiliza `FileReader` y `BufferedReader` para leer el contenido de los ficheros. Deberá leer el título del libro (que estará en la primera línea) y la descripción completa (que estará en las siguientes líneas).
3. Escribir y modificar ficheros de texto:
 - El programa debe permitir modificar la descripción de un libro. Para ello, debes leer el contenido de un fichero, modificar la descripción del libro (por ejemplo, cambiando una palabra clave) y luego guardar los cambios en el mismo fichero o en un nuevo fichero.
4. Copiar el contenido de un fichero a otro:
 - El programa debe copiar el contenido de un fichero que representa un libro a otro fichero. Para ello, utiliza `FileReader` para leer el fichero de origen y `FileWriter` para escribir el contenido en un nuevo fichero.
5. Realizar operaciones de gestión de ficheros:
 - Renombrar un fichero: El programa debe permitir cambiar el nombre de un fichero de un libro (por ejemplo, cambiar el nombre de un libro por una nueva edición).
 - Mover un fichero: El programa debe permitir mover un fichero de un libro desde su ubicación original a un nuevo directorio de almacenamiento.
 - Eliminar un fichero: El programa debe eliminar un fichero de libro si el usuario lo solicita. El programa debe comprobar si el fichero existe antes de eliminarlo.
 - Listar los ficheros de un directorio: El programa debe listar todos los ficheros de libros que se encuentran en el directorio de almacenamiento.

El diario mágico de la hechicera

En el corazón del bosque encantado, la hechicera Elara guarda un diario mágico que contiene los secretos de sus hechizos y conjuros. Tu misión es crear un programa en Java que gestione este diario, permitiendo la creación, lectura, modificación y eliminación de entradas.

1. El diario de Elara

Crea una clase `DiarioMagico` que represente el diario de la hechicera. Esta clase deberá manejar un fichero llamado `diario.txt` y poseer los siguientes métodos:

- `crearDiario()` → Crea el fichero si aún no existe.
- `escribirEntrada(String entrada)` → Añade una nueva entrada al diario sin borrar las anteriores.
- `leerDiario()` → Muestra el contenido del diario con un mensaje similar a:

"Elara ha escrito X entradas en su diario mágico."

- `modificarEntrada(int indice, String nuevaEntrada)` → Modifica una entrada existente del diario.
- `eliminarEntrada(int indice)` → Elimina una entrada del diario previa confirmación del usuario.

2. La aprendiz de hechicera

Crea una clase `AprendizHechicera` cuya función sea gestionar las entradas del diario antes de ser añadidas o modificadas. Para ello, hará uso de:

- Un `ArrayList` que contendrá las entradas pendientes de ser añadidas al diario.
- Un `HashMap<Integer, String>` que almacenará las entradas con su identificador único.

Dicha clase deberá contar con los siguientes métodos:

- `añadirEntrada(String entrada)` → Agrega una entrada al `ArrayList` y al `HashMap`.
- `listarEntradas()` → Muestra todas las entradas con su identificador.
- `guardarEntradasEnDiario()` → Escribe en el fichero todas las entradas del `ArrayList`, conservando su contenido previo.

3. El diario en acción

En el `main`, el programa deberá permitir lo siguiente:

- Verificar si el diario existe y crearlo si es necesario.
- Permitir al usuario añadir entradas al `ArrayList` y al `HashMap`.
- Guardar estas entradas en el fichero sin perder las anteriores.
- Leer el contenido del diario y mostrarlo en pantalla con el mensaje indicado.
- Brindar la opción de modificar o eliminar entradas si el usuario así lo desea.

Consejos adicionales

- Puedes utilizar la clase `Scanner` para obtener la entrada del usuario.
- Puedes utilizar la clase `File` para verificar si el fichero existe.
- Puedes utilizar la clase `FileWriter` para escribir en el fichero.
- Puedes utilizar la clase `BufferedReader` para leer el fichero.

¡Que la magia de la programación te acompañe en este desafío!

Ejercicio Completo: Gestión de un Sistema de Inventario

Descripción: Vas a crear un programa de gestión de inventario que almacena y maneja información sobre productos. Los productos deben almacenarse en un fichero de texto. El programa permitirá al usuario realizar varias operaciones como agregar nuevos productos, ver el inventario actual, buscar un producto específico, y guardar el inventario actualizado en el mismo fichero.

Requisitos:

1. El programa debe utilizar un fichero de texto para guardar el inventario de productos.
2. Cada producto tiene los siguientes atributos:
 - **ID** (número entero único)
 - **Nombre** (cadena de texto)
 - **Cantidad** (número entero)
 - **Precio** (número decimal)
3. El programa debe ser capaz de realizar las siguientes operaciones:
 - **Agregar un producto:** Añadir un nuevo producto al inventario.
 - **Ver el inventario:** Mostrar todos los productos en el inventario (ID, Nombre, Cantidad, Precio).
 - **Buscar un producto por ID:** Buscar un producto en el inventario por su ID.
 - **Guardar el inventario en el fichero:** Guardar el inventario actualizado en el fichero.
4. El programa debe permitir al usuario elegir entre las opciones de operación, y todas las opciones deben ser persistentes, es decir, si se cierra el programa y luego se vuelve a abrir, los datos no deben perderse.

Examen de Java: Manejo de Ficheros con POO y Colecciones – "Gestor de Coches"

Instrucciones

- Implementa un programa en Java utilizando Programación Orientada a Objetos (POO), ficheros de texto y colecciones.
 - Usa `BufferedReader`, `BufferedWriter`, `FileReader` y `FileWriter` correctamente.
 - Sigue buenas prácticas de programación.
 - No copies ni hagas chapuzas, que aquí se premia el esfuerzo y la lógica bien aplicada.
-

Ejercicio no Único: Gestor de Coches

Vas a desarrollar un programa que gestione una base de datos sencilla de coches utilizando ficheros de texto. Para ello, sigue estos pasos:

Crea una clase Coche con los atributos:

- id (entero, identificador único del coche)
 - marca (String)
 - modelo (String)
 - anio (int, año de fabricación)
 - precio (double)
2. **Crea una clase GestorCoches** que administre una colección de coches con un `ArrayList<Coche>`. Debe incluir métodos para:
- **Añadir un coche** a la colección (asegurando que el ID es único).
 - **Listar todos los coches** con sus datos.
 - **Buscar un coche por ID** y mostrar sus datos.
 - **Eliminar un coche** de la colección.
3. **Crea una clase ArchivoCoches** para manejar un fichero `coches.txt`. Debe tener métodos para:
- **Guardar todos los coches** en el fichero, cada coche en una línea con formato CSV (`id,marca,modelo,anio,precio`).
 - **Leer el fichero y cargar los coches** en la colección al iniciar el programa.
4. **Implementa en el main** la funcionalidad para:
- **Cargar los datos** desde el fichero al iniciar.
 - **Permitir al usuario interactuar** mediante un menú que permita añadir, listar, buscar y eliminar coches.
 - **Guardar automáticamente** en el fichero cuando haya cambios en la colección.
 - **Mostrar un mensaje con la cantidad de coches almacenados** en el fichero al cerrarlo.

Ejercicio Único: La Tienda de Libros

Imagina que eres el encargado de gestionar una tienda de libros y necesitas llevar un registro de los libros disponibles, junto con sus precios y autores. Además, deberás guardar estos datos en un fichero para su posterior consulta. A continuación, te planteo los pasos para hacerlo:

Requisitos del programa:

1. Clase **Libro**:

- **Responsabilidad:** Representar la entidad "Libro", con los siguientes atributos:
 - `titulo` (String)
 - `autor` (String)
 - `precio` (double)
- Debe tener un **constructor** que inicialice estos tres atributos.
- Además, debe tener un **método** `toString()` que devuelva una representación legible del libro (por ejemplo, "El Quijote de Cervantes - 25.50€").

2. Clase **TiendaDeLibros**:

- **Responsabilidad:** Gestionar la colección de libros en la tienda.
- Esta clase debe contener:
 - Un `ArrayList<Libro>` para almacenar los libros disponibles.
 - Un `HashMap<Integer, Libro>` para asociar un identificador único a cada libro.
 - Métodos para:
 - **Añadir un libro** al `ArrayList` y al `HashMap`.
 - **Mostrar los libros** en la tienda (debe imprimir el identificador, título, autor y precio de cada libro).
 - **Guardar los libros en un fichero** de texto. Cada línea debe contener los datos de un libro (título, autor y precio). El fichero debe llamarse `libros.txt`. Si el fichero ya existe, **no debe sobrescribir** el contenido anterior, sino añadir los nuevos libros al final.
 - **Leer los libros** del fichero `libros.txt` y cargarlos en el `ArrayList` y el `HashMap` al iniciar el programa. Los libros deben ser leídos desde el fichero línea por línea y convertirlos en objetos de tipo `Libro`.

3. En el **main**:

- Crear una instancia de `TiendaDeLibros`.
- Añadir varios libros a la tienda usando el método de la clase `TiendaDeLibros`.

- Guardar la lista de libros en el fichero `libros.txt`.
- Leer el contenido de `libros.txt` y cargar los libros en la tienda.
- Mostrar los libros cargados desde el fichero en pantalla.
- **Borrar el fichero** con confirmación del usuario si así lo desea.

Notas importantes:

- El programa debe manejar correctamente los errores de lectura y escritura de ficheros.
- La clase `Libro` debe ser sencilla, pero con todos los atributos y métodos necesarios para representar un libro de forma clara.
- La clase `TiendaDeLibros` debe ser capaz de gestionar la colección de libros, añadir nuevos libros y mantener la persistencia de los datos en el fichero `libros.txt`.
- Se debe implementar el uso de colecciones (`ArrayList` y `HashMap`) de manera adecuada para almacenar y manipular los libros.

Escribe un programa que lea un fichero de texto y genere un nuevo fichero con las palabras del original ordenadas por frecuencia de aparición (de mayor a menor). Cada línea del fichero resultante debe contener una palabra y su cantidad de ocurrencias, separadas por dos puntos (palabra:5).

Requisitos:

1. Usar `BufferedReader`, `FileReader`, `BufferedWriter` y `FileWriter`.
2. Almacenar las frecuencias en un `HashMap<String, Integer>`.
3. Convertir las entradas del `HashMap` en un `ArrayList` para ordenarlas.
4. El nombre del fichero de salida debe ser el mismo que el original, añadiendo `_sorted` antes de la extensión (ejemplo: `texto.txt` → `texto_sorted.txt`).
5. Implementar la lógica principal usando POO (crear al menos una clase adicional con métodos dedicados).

Ejemplo de funcionamiento:

`java ContadorPalabrasFrecuencia texto.txt`

- Si `texto.txt` contiene:
 - `hola mundo hola java mundo java hola`
- El fichero `texto_sorted.txt` generado será:
 - `hola: 3`
 - `java: 2`
 - `mundo: 2`

Notas:

- Ignorar diferencias entre mayúsculas y minúsculas (tratar todas como minúsculas).
- Eliminar signos de puntuación adjuntos a las palabras (ejemplo: `"hola!"` → `"hola"`).

A continuación se presenta un **examen avanzado de Java** sobre Fórmula 1. Sí, lo has leído bien, **FÓRMULA 1**. Si crees que este examen es pan comido, prepárate para demostrar lo contrario (¡aunque sabemos que ya te crees el rey de la programación, ¿no?!). Este examen está diseñado para retar hasta a los cerebros menos brillantes (sí, incluso al tuyo, si es que tienes uno) mediante el uso de archivos, ArrayList, POO, bucles y condiciones. Así que, siéntete afortunado de tener la oportunidad de lucirte, o al menos de intentarlo sin que tu código se deshaga en mil pedazos.

Examen Avanzo de Java: Simulación y Gestión de Datos en Fórmula 1

Instrucciones generales (para aquellos que no necesitan que se los expliquen todo a cada paso)

- **Objetivo:** Desarrollar una aplicación en Java que simule carreras de Fórmula 1 y gestione la información de los pilotos y resultados, utilizando clases, ArrayList, bucles, condiciones y manejo de archivos. Sí, ya sabemos que te cuesta entender instrucciones complejas, pero aquí vamos de nuevo.
 - **Organización:** Divide tu código en paquetes según la funcionalidad. Por ejemplo:
 - `modelo` para las clases de POO.
 - `persistencia` para las clases/métodos de lectura y escritura de archivos.
 - `aplicacion` o `principal` para la clase con el método `main` y la lógica de ejecución.
 - **Documentación:** Comenta tu código explicando las decisiones de diseño. O al menos intenta hacerlo; no quisiéramos que tu proyecto parezca escrito por alguien que se despertó tarde.
 - **Compilación:** Asegúrate de que el proyecto compile sin errores y se ejecute correctamente. Sí, ¡sorpréndenos y demuestra que no eres completamente inútil!
-

Pregunta 1: Diseño de la Clase Piloto

1. Diseña una clase `Piloto` con los siguientes atributos:
 - `nombre` (String)
 - `equipo` (String)
 - `numero` (int)
 - `nacionalidad` (String)
 - `puntos` (int)

2. Requerimientos para la clase:

- Implementa un **constructor** que inicialice todos los atributos (sí, incluso tú sabes cómo se hace un constructor, ¿no?).
- Crea los **métodos getters y setters** para cada atributo. Si esto te resulta demasiado básico, al menos inténtalo sin copiar y pegar.
- Sobrescribe el método `toString()` para mostrar la información completa del piloto. No queremos que se vea como un mensaje de error de novato.
- Sobrescribe el método `equals(Object o)` para considerar dos pilotos iguales si tienen el mismo **numero** y **nombre**. ¡Vamos, que no se te escape ningún detalle!
- Haz que la clase implemente la interfaz `Comparable<Piloto>` para que los pilotos puedan ordenarse de mayor a menor según el atributo **puntos**.
Pista: Recuerda sobrescribir también el método `compareTo` (sí, ya sabemos que la comparación te suena a chino, pero dale una oportunidad).

3. Comentario de diseño:

Explica brevemente en comentarios por qué decidiste implementar cada método y cómo piensas que la implementación de `equals` y `compareTo` facilitará la gestión de la lista de pilotos. Intenta que tus explicaciones sean un poco más brillantes que el promedio.

Pregunta 2: Simulación de una Carrera

1. Crea una clase **Carrera** que tenga un `ArrayList<Piloto>` para representar a los pilotos participantes.
(Sí, esos que has visto en el archivo... ¡y que por alguna razón te resultan difíciles de manejar!)
2. Implementa los siguientes métodos en la clase **Carrera**:
 - `void agregarPiloto(Piloto p):`
Añade un piloto al `ArrayList`, pero verifica (usando condiciones) que no se agregue un piloto con el mismo **numero**. No queremos duplicados, a menos que te guste el caos.
 - `void simularCarrera():`
Simula el resultado de una carrera asignando a cada piloto una posición de llegada de forma **aleatoria**.
 - Utiliza la clase `Random` para asignar posiciones (sí, esa clase que probablemente aún no dominas).
 - Según la posición de llegada, asigna puntos a los pilotos de la siguiente forma:
 - 1º lugar: 25 puntos

- 2º lugar: 18 puntos
 - 3º lugar: 15 puntos
 - 4º lugar: 12 puntos
 - 5º lugar: 10 puntos
 - 6º lugar: 8 puntos
 - 7º lugar: 6 puntos
 - 8º lugar: 4 puntos
 - 9º lugar: 2 puntos
 - 10º lugar: 1 punto
 - Actualiza el atributo **puntos** de cada piloto de acuerdo a su posición. Sorpréndenos y asegúrate de que no dejes ningún piloto en la banca.
 - Después de asignar los puntos, utiliza un **bucle** para ordenar el ArrayList de pilotos de forma **descendente** según sus puntos.
Sugerencia: Puedes utilizar **Collections.sort()** si implementaste **Comparable**. Si no lo hiciste, pues prepárate para ver el desastre.
 - Imprime en consola el **podio** (los tres primeros lugares). Queremos ver quiénes son los verdaderos campeones, o al menos a los que no se han quedado dormidos en el proceso.
3. **Recomendación:**
Utiliza bucles y condiciones donde sea necesario para recorrer y validar los datos. Sí, incluso tú, que a veces olvidas cómo se usan los bucles, ¡inténtalo!
-

Pregunta 3: Lectura de Archivos – Cargar Datos de Pilotos (Formato Modificado)

Prepara un archivo de texto llamado **pilotos.txt** con información de los pilotos en un formato sencillo.

Formato: Cada línea contendrá los datos de un piloto separados por **punto y coma (;)**.

Ejemplo de línea:

Lewis Hamilton;Mercedes;44;Reino Unido;0

1. *Nota:* Este formato es mucho menos complicado que el CSV, para que no te compliques la vida.
2. **Crea una clase llamada **GestorPilotos** (o similar) con un método que:**
 - Lea el archivo **pilotos.txt** línea por línea.
 - Por cada línea, utilice el método **split(";")** para separar los campos y cree un objeto **Piloto**. No te vayas a equivocar, que el split no es magia.
 - Almacene los objetos **Piloto** en un **ArrayList<Piloto>**.

- Imprima en consola la lista completa de pilotos leída del archivo, para que al menos se note que algo has hecho.
3. **Manejo de excepciones:**
- Utiliza bloques `try-catch` para capturar `FileNotFoundException` y `IOException`. Sí, porque aunque pienses que el mundo es perfecto, la realidad es que siempre hay errores (y probablemente sean culpa tuya).
 - En caso de error, muestra un mensaje descriptivo en la consola. Nada de dejar al usuario adivinando lo que hiciste mal.
-

Pregunta 4: Escritura de Archivos – Guardar Resultados de una Carrera

En la clase `Carrera`, implementa un método:

`void guardarResultados(String nombreArchivo)`

1. que haga lo siguiente:
 - Escriba en el archivo (por ejemplo, `resultados.txt`) los resultados de la carrera.
 - Cada línea del archivo debe contener: posición, nombre del piloto, equipo y puntos obtenidos en la carrera.
 - Utiliza un bucle para recorrer la lista de pilotos (ya ordenada) y escribir la información. No, no esperes que la magia suceda sola.
 - Asegúrate de manejar las excepciones necesarias durante la escritura del archivo (por ejemplo, `IOException`). Recuerda: ¡tu código no es perfecto (ni se supone que lo sea)!
-

Pregunta 5: Simulación de una Temporada Completa

1. **Crea una clase `Temporada` que simule una temporada de Fórmula 1 compuesta por 5 carreras.**
(Sí, cinco carreras. Ojalá tu capacidad de organización sea mayor que la de un principiante de siempre).
2. **Requerimientos de la clase `Temporada`:**
 - En el constructor o en un método de inicialización, crea o carga una lista de pilotos (puedes reutilizar el método de lectura de `GestorPilotos`). Esperamos que esta vez no olvides invocar el método.

- Para cada una de las 5 carreras, crea una instancia de **Carrera** y:
 - Agrega los mismos pilotos a cada carrera. No, no queremos que empieces a inventar pilotos nuevos en cada carrera.
 - Invoca el método **simularCarrera()** para simular el resultado. Si logras que funcione, te felicitamos (aunque sea poco).
 - Acumula los puntos obtenidos por cada piloto a lo largo de las carreras. Sí, que no se te escape ni un punto.
 - Al final de la temporada, imprime en consola la **clasificación final** (ordenada de mayor a menor según los puntos totales). Queremos ver quién se llevó el mérito (o quién se las apañó mejor).
 - Guarda el resultado final en un archivo llamado **clasificacion_final.txt**. Ojalá no lo dejes a medio hacer.
3. **Recomendación:**
Utiliza bucles anidados y condiciones para gestionar la simulación de cada carrera y la acumulación de puntos. No, no es tan complicado, incluso tú deberías poder hacerlo sin llorar.
-

Pregunta 6: Búsqueda de Pilotos y Manejo de Condiciones

En la clase **Temporada**, implementa el método:

```
void buscarPiloto(int numero)
```

1. que realice lo siguiente:
 - Reciba como parámetro el **numero** de un piloto.
 - Busque en la lista de pilotos si existe un piloto con ese número. Ojo, que no se te escape ningún error por distracción.
 - Si se encuentra, muestra en consola el rendimiento del piloto a lo largo de la temporada, por ejemplo, los puntos obtenidos en cada carrera (puedes almacenar esta información en una estructura auxiliar o mostrar la información acumulada). Sorpréndenos, que no es tan difícil.
 - Si el piloto no se encuentra, muestra un mensaje adecuado indicando que no existe. Así, al menos, el usuario sabrá que no inventaste datos.
 - Utiliza condiciones para verificar la existencia del piloto en la lista. ¡Sí, esas condiciones que te han costado tanto entender!
-

Pregunta 7: Reflexión y Extensiones (para los que aún creen que han terminado)

1. Optimización en el Manejo de Archivos:

- En comentarios o en un breve documento anexo dentro del código, explica cómo podrías optimizar la lectura y escritura de archivos para trabajar con grandes volúmenes de datos. Por ejemplo, ¿qué técnicas (buffers, streams, librerías de terceros) o patrones de diseño implementarías? Intenta que no parezca que lo hiciste a la última hora (porque probablemente lo hiciste).

2. Extensión de la Simulación:

- Imagina que se desea agregar la funcionalidad de simular **penalizaciones** o **bonificaciones** durante una carrera (por ejemplo, por infracciones o por superar a un oponente).
 - ¿Qué modificaciones harías en el diseño de tus clases (**Piloto**, **Carrera**, **Temporada**)?
 - Describe brevemente (en comentarios) cómo integrarías estas nuevas funcionalidades sin romper la arquitectura actual. (O al menos, sin hacer un lío que ni tú mismo puedas arreglar después.)
 - Si lo deseas, implementa un ejemplo sencillo de penalización o bonificación. ¡Vamos, sorpréndenos de una vez!

Requisitos Adicionales

- **Claridad y Estilo:**

Tu código debe ser legible, con nombres significativos para variables y métodos. Aunque, seamos sinceros, si no cuidas estos detalles, es el reflejo de tu desorden mental.

- **Validación:**

Considera la validación de datos tanto en la lectura de archivos como en la entrada de datos (si fuera el caso). No queremos que tu programa se caiga porque no pensaste en todo.

- **Excepciones:**

Maneja correctamente todas las excepciones posibles en operaciones de I/O. Sí, incluso aquellas que creías insignificantes.

- **Pruebas:**

Realiza pruebas con diferentes archivos y escenarios para asegurarte de que la aplicación funciona correctamente. O al menos, intenta que no se rompa todo a la primera.

Este examen pone a prueba la integración de múltiples conceptos de Java en una aplicación completa. Si logras completarlo sin que tu código se convierta en un chiste, ¡felicidades! Y si no, pues siempre puedes culpar a la complejidad del lenguaje (aunque sepas muy bien que la culpa es tuya).

Consejo irónico: Al utilizar el delimitador de punto y coma, el método `split(";")` te permitirá separar fácilmente los campos de cada línea, facilitando la conversión de esos valores a los tipos de datos adecuados (por ejemplo, convertir el campo `numero` y `puntos` a `int` usando `Integer.parseInt(campo)`). Sí, incluso tú puedes hacerlo si te concentras (o al menos lo intentas).

¡Buena suerte programando, y que al menos este examen no te haga quedar peor de lo que ya sospechamos!

Registro de Participantes en un Evento Deportivo

Desarrolla una aplicación en Java que permita registrar y gestionar participantes de un evento deportivo utilizando **colecciones** y **manejo de archivos**.

Requisitos:

1. Lectura de Datos

- La aplicación debe leer un archivo de texto (`participantes.txt`) usando `BufferedReader`.

Cada línea del archivo representa un participante con el siguiente formato:

`ID,Nombre,Edad,Deporte,TiempoRegistro`

`101,Juan Pérez,25,Atletismo,12.5`

`102,María López,22,Natación,50.3`

`103,Carlos García,28,Ciclismo,120.8`

- Los participantes deben almacenarse en una **colección** (`HashMap<Integer, Participante>`), donde la clave es el **ID**.

2. Funciones principales:

- **Listar todos los participantes** y sus datos.
- **Buscar un participante** por ID o por nombre.
- **Registrar un nuevo participante** en el evento y guardar los cambios con `BufferedWriter`.
- **Eliminar un participante** del sistema y actualizar el archivo.
- **Actualizar el tiempo de registro de un participante** tras una nueva competencia.

3. Manejo de Archivos

- Cada modificación (agregar, eliminar, actualizar) debe reflejarse en `participantes.txt`.
- Si el archivo no existe, debe crearse automáticamente usando la clase `File`.

4. Interfaz de Usuario (Consola)

- Implementar un **menú interactivo** donde el usuario pueda elegir opciones.

El Gestor de Tareas Pendientes de Isaac

Isaac quiere organizar sus tareas diarias mediante un programa que le permita registrar, modificar, eliminar y visualizar las tareas que tiene pendientes. Tu misión es crear un programa en Java que gestione su lista de tareas y permita manipularlas mediante un archivo.

Parte 1: El Gestor de Tareas

1. Crea una clase **GestorTareas** que represente el gestor de tareas de Isaac. Esta clase debe manejar un fichero llamado **tareas.txt** y tener los siguientes métodos:
 - **crearLista()** → Crea el fichero **tareas.txt** si aún no existe.
 - **agregarTarea(String tarea)** → Añade una nueva tarea al archivo sin borrar las anteriores.
 - **verTareas()** → Muestra todas las tareas almacenadas en el archivo.
 - **modificarTarea(int indice, String nuevaTarea)** → Modifica una tarea existente en la lista.
 - **eliminarTarea(int indice)** → Elimina una tarea de la lista previa confirmación del usuario.
-

Parte 2: El Asistente de Tareas

2. Crea una clase **AsistenteTareas** cuya función será gestionar las tareas antes de ser añadidas o modificadas en el gestor. Para ello, hará uso de:
 - Un **ArrayList** que contendrá las tareas pendientes de ser añadidas a la lista.
 - Un **HashMap<Integer, String>** que almacenará las tareas con su identificador único.
3. Esta clase debe contar con los siguientes métodos:
 - **añadirTarea(String tarea)** → Agrega una nueva tarea al **ArrayList** y al **HashMap**.
 - **listarTareas()** → Muestra todas las tareas con su identificador único.

- **guardarTareasEnLista()** → Escribe todas las tareas pendientes en el archivo `tareas.txt`, sin perder las tareas anteriores.
-

Parte 3: El Gestor en Acción

3. En el método `main`, el programa deberá permitir lo siguiente:
 - Verificar si el archivo de tareas (`tareas.txt`) existe y crearlo si es necesario.
 - Permitir al usuario añadir nuevas tareas al `ArrayList` y al `HashMap`.
 - Guardar estas tareas en el archivo sin perder las anteriores.
 - Leer el contenido de la lista de tareas y mostrarlo en pantalla.
 - Ofrecer la opción de modificar o eliminar tareas si el usuario lo desea.
-

Consejos adicionales:

1. **Crear el archivo:** Usa la clase `File` para verificar si el fichero `tareas.txt` existe y, si no, crearlo.
2. **Leer y escribir en el archivo:** Utiliza `BufferedReader` para leer el contenido del archivo y `FileWriter` o `BufferedWriter` para escribir las nuevas tareas.
3. **Gestión de tareas:** La clase `AsistenteTareas` gestionará la adición y modificación de tareas antes de guardarlas definitivamente en la lista, usando colecciones como `ArrayList` y `HashMap`.
4. **Interacción con el usuario:** Usa la clase `Scanner` para interactuar con el usuario y permitirle gestionar la lista de tareas, añadiendo, modificando o eliminando entradas.

Examen de Java: Manejo de Ficheros de Texto, POO, Colecciones, ArrayList, HashMap y Genéricos

Contexto del Minijuego:

Bienvenido a "El Rescate de los Manuscritos Perdidos". El sistema de archivos de una gran biblioteca digital se ha corrompido, dejando miles de archivos de texto desorganizados y duplicados. Tu misión como desarrollador es rescatar y clasificar estos archivos, garantizando el orden y la búsqueda eficiente.

Cada fase representa un reto que deberás superar con tu conocimiento de Java. Completa todas las fases y obtén los 100 puntos disponibles. Hay recompensas por tareas extra.

Fase 1: Exploración de Manuscritos (20 puntos)

Objetivo: Leer archivos de texto desde un directorio y modelarlos en una clase `Texto`.

Descripción:

Tienes acceso a una carpeta llamada `documentos/` que contiene varios archivos `.txt`. Tu misión es leer cada archivo de texto y almacenarlo como un objeto en una lista.

Requisitos:

1. Crear la clase `Texto` con los siguientes atributos privados:
 - `nombre` (String): nombre del archivo (sin la extensión)
 - `contenido` (String): contenido completo del archivo
2. Leer cada archivo de la carpeta `documentos/` y crear un objeto `Texto` para cada archivo, almacenándolos en un `ArrayList<Texto>`.
3. Mostrar en consola los nombres de los archivos leídos.

Ejemplo de salida esperada:

Archivos cargados:

- poema
- informe
- notas

Puntos Extra (5 puntos): Validar que la carpeta `documentos/` existe y manejar posibles excepciones de lectura.

Fase 2: Clasificación por Palabras Clave (25 puntos)

Objetivo: Organizar los textos en un `HashMap` según una palabra clave que contengan.

Descripción:

El sistema necesita que clasifiques los archivos según una palabra clave presente en su contenido. Las palabras clave son: `importante`, `revisión`, `borrador`. Si un archivo contiene alguna de estas palabras, se clasifica bajo esa categoría.

Requisitos:

1. Crear un método `clasificarTextos` que reciba un `ArrayList<Texto>` y devuelva un `HashMap<String, ArrayList<Texto>>`.
2. Mostrar en consola los textos clasificados por palabra clave.

Ejemplo de salida esperada:

Categoría: `importante`

- `informe`

Categoría: `revisión`

- `notas`

Categoría: `borrador`

- `poema`

Puntos Extra (5 puntos): Permitir que el usuario ingrese nuevas palabras clave para clasificar los archivos.

Fase 3: Eliminación de Duplicados (30 puntos)

Objetivo: Detectar y eliminar archivos de texto duplicados.

Descripción:

Algunos archivos son duplicados porque contienen exactamente el mismo contenido, aunque su nombre sea distinto. Tu misión es identificar estos duplicados y eliminarlos.

Requisitos:

1. Implementar un método `eliminarDuplicados` que reciba un `ArrayList<Texto>` y elimine los archivos que tengan contenido duplicado.
2. Mostrar en consola los nombres de los archivos eliminados.

Ejemplo de salida esperada:

Se eliminaron los siguientes archivos duplicados:

- notas_copia
- informe_v2

Puntos Extra (5 puntos): Guardar el listado limpio de archivos en un nuevo directorio `limpios/`, creando un archivo de texto por cada entrada restante.

Fase 4: Búsqueda Genérica de Textos (25 puntos)

Objetivo: Implementar una clase genérica para buscar textos según diferentes criterios.

Descripción:

El sistema requiere una herramienta de búsqueda flexible para encontrar textos. Debes crear una clase genérica `Buscador<T>` que permita buscar textos por nombre o contenido.

Requisitos:

1. Implementar la clase genérica `Buscador<T>`, que tenga un método `buscar(ArrayList<T> lista, T criterio)` que devuelva los elementos que coincidan con el criterio.
2. Aplicar el `Buscador` para buscar textos en tu `ArrayList<Texto>`.

Ejemplo de uso:

- Buscar archivos cuyo nombre contenga "informe".
- Buscar archivos cuyo contenido contenga la palabra "borrador".

Puntos Extra (5 puntos): Permitir búsquedas compuestas (por nombre y contenido simultáneamente).

Evaluación y Entrega

El puntaje máximo es de 100 puntos, con 20 puntos extra opcionales.
¡Buena suerte en tu misión para salvar los manuscritos perdidos! 🚀

Ejercicio de Examen

Realiza un programa en Java que analice un fichero de texto y genere un nuevo fichero con un resumen del contenido.

El programa debe:

1. Leer un fichero de texto cuyo nombre se pasará como argumento en la línea de comandos.
2. Contar el número de líneas, palabras y caracteres en el fichero.
3. Guardar estos datos en un nuevo fichero con el mismo nombre que el original, pero añadiendo la coletilla `_resumen.txt`.
4. Mostrar por pantalla un mensaje indicando que el resumen ha sido generado correctamente.

Detalles adicionales:

- Una palabra se define como cualquier secuencia de caracteres separada por espacios en blanco.
- Se debe utilizar `BufferedReader` para la lectura del fichero y `BufferedWriter` para escribir el resumen.
- Si el fichero no existe o no se puede leer, el programa debe mostrar un mensaje de error adecuado.

Ejemplo de uso:

```
java AnalizaTexto documento.txt
```

Si `documento.txt` contiene:

```
Hola, esto es un archivo de prueba.
```

```
Tiene varias líneas de texto.
```

```
Cada línea tiene varias palabras.
```

El programa debe generar un archivo llamado `documento_resumen.txt` con el siguiente contenido:

```
Número de líneas: 3
```

```
Número de palabras: 16
```

```
Número de caracteres: 96
```

Puntos a evaluar:

- ✓ Uso correcto de `BufferedReader` y `BufferedWriter`.
- ✓ Manejo de errores y excepciones.

- ✓ Manipulación de cadenas y contadores.
- ✓ Generación correcta del fichero resumen.